

Creature Crafter — Build & Run Instructions

Prerequisites

- **Visual Studio Community 2022** (free) — <https://visualstudio.microsoft.com/vs/community/>
 - During install, select the **.NET desktop development** workload (includes .NET 8 and WPF support)
-

Step 1 — Create the project

1. Open Visual Studio 2022
 2. Click **Create a new project**
 3. Search for **WPF Application**, select the **C#** version, click **Next**
 4. Set **Project name:**
 5. Choose your save location, click **Next**
 6. Set **Framework:** , click **Create**
-

Step 2 — Add all files

You need the following files. Files marked **(replace)** already exist — open them and replace all content. Files marked **(add)** need to be created fresh.

File	Action	How to add
<code>MainWindow.xaml</code>	Replace	Already exists
<code>MainWindow.xaml.cs</code>	Replace	Already exists
<code>Models.cs</code>	Add	Right-click project → Add → Class → name it <code>Models.cs</code>
<code>EditScreen.xaml</code>	Add	Right-click project → Add → New Item → User Control (WPF) → <code>EditScreen.xaml</code>
<code>EditScreen.xaml.cs</code>	—	Auto-created with above
<code>HistoryScreen.xaml</code>	Add	Right-click project → Add → New Item → User Control (WPF) → <code>HistoryScreen.xaml</code>
<code>HistoryScreen.xaml.cs</code>	—	Auto-created with above
<code>ToggleSwitch.xaml</code>	Add	Right-click project → Add → New Item → User Control (WPF) → <code>ToggleSwitch.xaml</code>
<code>ToggleSwitch.xaml.cs</code>	—	Auto-created with above
<code>InputDialog.xaml</code>	Add	Right-click project → Add → New Item → Window (WPF) → <code>InputDialog.xaml</code>
<code>InputDialog.xaml.cs</code>	—	Auto-created with above
<code>ResultDetailDialog.xaml</code>	Add	Right-click project → Add → New Item → Window (WPF) → <code>ResultDetailDialog.xaml</code>
<code>ResultDetailDialog.xaml.cs</code>	—	Auto-created with above
<code>CreatureCrafter.csproj</code>	Replace	Open in Solution Explorer → right-click project → Edit Project File

Tip: When Visual Studio creates a User Control or Window it inserts boilerplate — always **select all and replace** with the provided code rather than merging.

Step 3 — Paste the code

For each file, copy the full content from the matching artifact and paste it into Visual Studio,

replacing whatever was already there.

Important for `.xaml` files: Visual Studio auto-generates an `x:Class` attribute at the top. Make sure it reads exactly:

- `x:Class="CreatureCrafter.EditScreen"` etc. — matching the class in the paired `.xaml.cs`
-

Step 4 — Build

Press **Ctrl+Shift+B**. The Output window should say **Build succeeded**.

Common errors and fixes:

Error	Fix
<code>The name 'X' does not exist in the current context</code>	Make sure the <code>.xaml.cs</code> namespace is <code>CreatureCrafter</code> and class name matches <code>x:Class</code> in the <code>.xaml</code>
<code>Could not find type 'CreatureCrafter.ToggleSwitch'</code>	Ensure <code>ToggleSwitch.xaml</code> was added as a User Control (WPF) , not a plain class
<code>CS0246: type not found</code>	Check that all files are in the project (show in Solution Explorer) and namespace is <code>CreatureCrafter</code>

Step 5 — Run

Press **F5** (or click the green ► button).

- **First launch:** No save file exists, so you land directly on the **Edit Categories** screen. Add some categories and options, then press **Back** to reach the main screen.
 - **Subsequent launches:** Your data, last result, and window size/position are all restored automatically.
-

Save file location

Your data is saved as `creature_crafter_data.json` in the same folder as the executable:

```
<your project folder>\bin\Debug\net8.0-windows\creature_crafter_data.json
```

To reset all data, simply delete this file.